Übungsblatt 9: C, Assembler und Lüfterregler

Mikrocomputertechnik 1

Musterlösung

Aufgabe 1: Vom Quelltext zum Programm (8 Punkte)

Ein Lüfterregler für einen Schaltschrank begleitet Sie durch das ganze Blatt. Er kennt drei Stufen: 0 aus, 1 langsam, 2 schnell. Bedient wird er über ein D-Pad (UP = PLUS, DOWN = MINUS, LEFT = STOP). Eine LED zeigt die Stufe: schwarz, grün (`0x0000FF00`) oder rot (`0x00FF0000`); ein Motor wird nicht angesteuert. Die Firmware besteht aus zwei C-Dateien: `regler.c` enthält die Hauptschleife und ruft `next_level` auf, `stufen.c` enthält diese Funktion. Der Header `stufen.h` deklariert sie:

```
int next_level(int stufe, int ereignis);
```

Gebaut wird mit einer RISC-V-Toolchain, die RV32 unterstützt. Der Aufruf für die erste Tabellenzeile lautet zum Beispiel:

```
riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 \
  -E regler.c -o regler.i
```

1. Ergänzen Sie die Tabelle. Sie nennt nur Stoppoption, Eingabe und Ausgabe; die erste Zeile ist ein Beispiel. (3 P)

Option	Eingabe	Ausgabe	Inhalt der Ausgabe	zuletzt beteiligt
`-E`	`regler.c`	`regler.i`	vorverarbeiteter C-Text	Präprozessor
`-S`	`regler.c`	`regler.s`		
`-c`	`regler.s`	`regler.o`		
keine	`regler.o`, `stufen.o`	`regler.elf`		

2. Jemand ruft `-S stufen.c -o stufen.o` auf. Was steht in `stufen.o`? Begründen Sie kurz. (1 P)
3. Ein Team linkt nur `regler.o` zu `regler.elf`. Der Linkschritt bricht ab, weil `next_level` nicht definiert sei. `regler.c` bindet `stufen.h` aber ein. Welche Stufe meldet den Fehler? Warum hilft die Deklaration im Header nicht, und wie beheben Sie den Fehler? (2 P)
4. Ein anderes Team ruft auf dem eigenen Laptop nur `gcc -S regler.c` auf. Der erzeugte Assemblertext enthält Befehle, die es in RV32I nicht gibt. Warum? Was leisten der andere Treibername und die beiden Optionen im Aufruf oben? (2 P)

Musterlösung

Option	Eingabe	Ausgabe	Inhalt der Ausgabe	zuletzt beteiligt
`-S`	`regler.c`	`regler.s`	Assemblertext	Compiler
`-c`	`regler.s`	`regler.o`	Objektdatei, noch nicht gelinkt	Assembler
keine	`regler.o`, `stufen.o`	`regler.elf`	gelinktes Programm	Linker

1. Je Zeile 1 P (Inhalt und Stufe je 0,5 P). Beim `-S`-Aufruf laufen Präprozessor und Compiler. Die Objektdatei enthält Maschinenbefehle, aber noch offene Symbole wie `next_level`. Der Linkschritt gelingt nur, wenn Eingaben und Zielumgebung passen; die Endung `.elf` allein beweist keine Ausführbarkeit.
2. Assemblertext: `-S` stoppt nach der Übersetzung, die frei gewählte Endung ändert den Inhalt nicht (1 P).
3. Der Linker (1 P). Der Header liefert nur die Deklaration, also die Schnittstelle. Den Maschinenkörper von `next_level` enthält erst `stufen.o`. Abhilfe: `stufen.c` mit `-c` übersetzen und `stufen.o` mitlinken (1 P).
4. Ein gewöhnliches `gcc` auf dem Arbeitsrechner (Host) erzeugt Code für dessen eigenes Ziel, nicht für RISC-V (1 P). Der RISC-V-Treiber erzeugt Code für das Ziel; `-march=rv32i` wählt die Ganzzahlbasis RV32I, `-mabi=ilp32` das Daten- und Aufrufmodell mit 32-Bit-int und -Zeigern (1 P).

Aufgabe 2: Der Lüfteranlauf als Schleife (10 Punkte)

Beim Einschalten erhöht der Regler die Solldrehzahl in Schritten von 300 U/min, bis das Ziel erreicht oder überschritten ist:

```
int anlauf(int drehzahl, int ziel) {
    while (drehzahl < ziel) {
        drehzahl += 300;
    }
    return drehzahl;
}
```

1. Verfolgen Sie `anlauf(0, 1000)`. Die erste Zeile ist ein Beispiel. (2 P)

Prüfung	drehzahl vor der Prüfung	drehzahl < ziel?	Wirkung
1	0	ja	drehzahl = 300
2			
3			
4			
5			

2. Wie oft läuft der Körper, wie oft wird geprüft, und was wird zurückgegeben? (1 P)
3. Ergänzen Sie die drei Lücken der Lehrübersetzung. drehzahl liegt in a0, ziel in a1, das Ergebnis in a0. Der Befehl bei (a) verlässt die Schleife, sobald drehzahl $\geq$ ziel gilt. (3 P)

```
anlauf:
anlauf_test:
    _______ a0, a1, anlauf_ende    # (a)
    addi    a0, a0, _______        # (b)
    j       _______                # (c)
anlauf_ende:
    ret
```

4. Was liefert `anlauf(1500, 1000)`? Wie oft wird geprüft? (1 P)

Ein Kollege hat bei (a) `bgeu` eingesetzt. Ein Luftzug dreht den stehenden Lüfter rückwärts, der Sensor meldet −600 U/min. `anlauf(-600, 1000)` liefert nun sofort −600, der Lüfter läuft nicht an.

5. Geben Sie das 32-Bit-Muster von −600 hexadezimal an. (1 P)
6. Warum bricht die Schleife mit `bgeu` schon bei der ersten Prüfung ab? (1 P)
7. Was liefert die Fassung mit dem richtigen Befehl, und wie oft läuft ihr Körper? (1 P)

Musterlösung

Prüfung	drehzahl vor der Prüfung	drehzahl < ziel?	Wirkung
2	300	ja	drehzahl = 600
3	600	ja	drehzahl = 900
4	900	ja	drehzahl = 1200
5	1200	nein	Rückgabe 1200

1. Je Zeile 0,5 P.
2. Vier Körperdurchläufe, fünf Prüfungen, Rückgabe 1200 (1 P). Die Schleife hört erst auf, wenn drehzahl $\geq$ ziel gilt; 1200 überschreitet das Ziel.
3. Bei (a) `bge`, der vorzeichenbehaftete Abbruch als Umkehrung von drehzahl < ziel (1 P). Bei (b) 300 (1 P). Bei (c) `anlauf_test`, die Rückkante zur Prüfung (1 P).
4. 1500 nach einer Prüfung, der Körper läuft nicht (1 P).
5. $2^{32} - 600 = 4\,294\,966\,696$ = `0xFFFFFDA8` (1 P).
6. `bgeu` vergleicht vorzeichenlos. Das Muster gilt dann als 4 294 966 696 und ist größer als 1000, der Abbruch greift sofort (1 P).
7. −600 → −300 → 0 → 300 → 600 → 900 → 1200: Rückgabe 1200 nach sechs Körperdurchläufen und sieben Prüfungen (1 P).

Aufgabe 3: Zuluft-Temperaturen im Messpuffer (10 Punkte)

Der Regler speichert Temperaturen in einem int-Array: an geraden Indizes die Zuluft, an ungeraden die Abluft. Im ILP32-Modell belegt ein int vier Byte. Der Puffer beginnt bei B = `0x5600` und enthält sechs Werte in °C:

Index	0	1	2	3	4	5
Wert	21	35	23	38	25	41

1. Ergänzen Sie die Tabelle. Die erste Zeile ist ein Beispiel. (2 P)

Index i	Byteoffset 4i	Adresse	Wert	Messung
1	4	`0x5604`	35	Abluft
2				
4				
5				
6				

Die Funktion `zuluft_summe` addiert nur die Zuluftwerte:

```
int zuluft_summe(const int *p, unsigned n) {
    int sum = 0;
    for (unsigned i = 0; i < n; i += 2) {
        sum += p[i];
    }
    return sum;
}
```

2. Ergänzen Sie die drei Lücken der Lehrübersetzung. Begründen Sie bei (a) die Wahl zwischen signed und unsigned. (3 P)

```
zuluft_summe:                  # a0 = p, a1 = n -> a0 = Summe
    mv    t0, a0               # Basis
    li    t1, 0                # Index i
    li    t2, 0                # sum
schleife:
    _____ t1, a1, ende         # (a) Abbruch, wenn i >= n
    slli  t3, t1, ___          # (b) Byteoffset
    add   t4, t0, t3
    lw    t5, 0(t4)
    add   t2, t2, t5
    addi  t1, t1, ___          # (c) nächster Zuluftindex
    j     schleife
ende:
    mv    a0, t2
    ret
```

3. Verfolgen Sie `zuluft_summe(B, 6)`. Die erste Zeile ist ein Beispiel. (2 P)

i	i < n?	Adresse	geladener Wert	sum danach
0	ja	`0x5600`	21	21
2				
4				
6				

4. Was liefert `zuluft_summe(B, 0)`, und wie viele Elemente werden dabei gelesen? (1 P)
5. Eine erste Fassung liefert für diesen Puffer 79 statt 69. Welche drei Werte wurden addiert, und an welchen Adressen liegen sie? Welche Lücke war falsch gefüllt? Tipp: 21 wird in jedem Fall richtig gelesen, und $79 - 21 = 58$. (2 P)
6. Optional, ohne Punkte: Übersetzen Sie `anlauf` aus Aufgabe 2 und `zuluft_summe` mit Compiler Explorer (godbolt.org, Sprache C, ein RISC-V-Compiler für 32 Bit) oder einer RISC-V-GNU-Toolchain, je mit `-O0` und `-O2` und den Zieloptionen aus Aufgabe 1. Protokollieren Sie Compiler, Version und Optionen. Welche Branch-Befehle prüfen die Schleifen, und wie schreitet die Schleife von `zuluft_summe` voran? Unterscheiden Sie gleiche Wirkung von gleicher Befehlsfolge.

Musterlösung

Index i	Byteoffset 4i	Adresse	Wert	Messung
2	8	`0x5608`	23	Zuluft
4	16	`0x5610`	25	Zuluft
5	20	`0x5614`	41	Abluft
6	24	`0x5618`	—	kein Element, nicht lesen

1. Je Zeile 0,5 P. `0x5618` liegt direkt hinter dem sechsten Element; dort steht kein Wert des Puffers.
2. Bei (a) `bgeu`, denn i und n sind unsigned (1 P). Bei (b) 2, denn ein int belegt $4 = 2^2$ Byte (1 P). Bei (c) 2, denn nur jeder zweite Index ist Zuluft (1 P). Die beiden Zweien haben verschiedene Gründe.

i	i < n?	Adresse	geladener Wert	sum danach
2	ja	`0x5608`	23	44
4	ja	`0x5610`	25	69
6	nein	—	—	Rückgabe 69

3. Je Zeile 2/3 P. Drei Loads und vier Prüfungen.
4. 0, und es wird kein Element gelesen: Schon die erste Prüfung $0 < 0$ ist falsch (1 P).
5. $58 = 35 + 23$: Addiert wurden 21, 35 und 23 an `0x5600`, `0x5604` und `0x5608`, also p[0], p[1] und p[2] (1 P). Für i = 2 und i = 4 entstanden die Offsets 4 und 8, also $2i$ statt $4i$: Bei (b) stand 1 statt 2 (1 P).
6. Jede korrekte Übersetzung hat dieselbe Wirkung wie die Lehrübersetzung: `anlauf` liefert für die Startwerte 0 und −600 mit Ziel 1000 jeweils 1200, `zuluft_summe` für den Puffer mit n = 6 den Wert 69. Die Prüfung von `anlauf` muss wie ein vorzeichenbehafteter Vergleich wirken, die von `zuluft_summe` wie ein vorzeichenloser. Ein optimierender Compiler kann etwa einen Zeiger direkt um 8 Byte weiterschalten, statt Index und Shift zu berechnen. Eine bestimmte Befehlsfolge wird nicht vorausgesagt; maßgeblich ist der protokollierte eigene Lauf.

Aufgabe 4: Stufen, Tasten und Ereignisse (8 Punkte)

Die Stufe ist der Zustand eines endlichen Zustandsautomaten; die Hauptschleife hält sie in s0. Die Blattfunktion `next_level` erhält die Stufe in a0 und das Ereignis in a1 und liefert die Folgestufe in a0. Ereignisse: 0 keines, 1 PLUS, 2 MINUS, 3 STOP. PLUS erhöht um eins bis höchstens 2, MINUS vermindert um eins bis mindestens 0, STOP setzt 0. Ohne Ereignis bleibt die Stufe. Ein ungültiger Stufenwert wird immer 0, auch ohne Ereignis.

1. Ergänzen Sie die Übergangstabelle mit den Folgestufen. Die erste Zeile ist ein Beispiel. (3 P)

Stufe	0 keines	1 PLUS	2 MINUS	3 STOP
0	0	1	0	0
1				
2				
17 (ungültig)				

Die Hauptschleife liest in jeder Abfragegruppe die drei Tasten einmal und bildet die Rohmaske UP = 1, DOWN = 2, LEFT = 4. Das Ereignis entsteht nach dieser Freigaberegel:

- Die Rohmaske 0 öffnet die Freigabe; ein Ereignis entsteht nicht.
- Jede andere Rohmaske schließt die Freigabe. Nur wenn die Freigabe vorher offen war und genau eine Taste gedrückt ist, entsteht ein Ereignis: Rohmaske 1 → PLUS (1), 2 → MINUS (2), 4 → STOP (3).
- Beim Start ist die Freigabe geschlossen.

2. Anfangs gelten Stufe 1 und Freigabe geschlossen (0). Ergänzen Sie die Tabelle. Die erste Zeile ist ein Beispiel. (4 P)

Gruppe	Rohmaske	Freigabe vorher	Ereignis	Stufe danach
1	0	0	0	1
2	1			
3	1			
4	0			
5	2			
6	0			
7	6			
8	4			
9	0			

3. In Gruppe 8 ist nur LEFT gedrückt, trotzdem entsteht kein STOP. Warum? Welche Rohmaske muss in Gruppe 10 folgen, damit der Lüfter ausgeht? (1 P)

Musterlösung

Stufe	0 keines	1 PLUS	2 MINUS	3 STOP
1	1	2	0	0
2	2	2	1	0
17 (ungültig)	0	0	0	0

1. Je Zeile 1 P. PLUS in Stufe 2 hält 2, MINUS in Stufe 0 hält 0.

Gruppe	Rohmaske	Freigabe vorher	Ereignis	Stufe danach
2	1	1	1 (PLUS)	2
3	1	0	0	2
4	0	0	0	2
5	2	1	2 (MINUS)	1
6	0	0	0	1
7	6	1	0	1
8	4	0	0	1
9	0	0	0	1

2. Je Zeile 0,5 P. In Gruppe 3 wird die Taste nur gehalten. In Gruppe 7 sind DOWN und LEFT zugleich gedrückt: kein Ereignis, aber die Freigabe ist verbraucht.
3. Die Mehrfachmaske 6 hat die Freigabe geschlossen; erst die leere Gruppe 9 öffnet sie wieder. Rohmaske 4 in Gruppe 10 erzeugt STOP (3) und Stufe 0 (1 P).

Aufgabe 5: Fehlersuche in der Hauptschleife (4 Punkte)

Die Hauptschleife aus Aufgabe 4 führt in jeder Abfragegruppe nach der Ereigniserkennung diesen Block aus, auch ohne Ereignis. s3 enthält die Basisadresse der LED-Matrix.

```
update:
    mv   a0, s0              # alte Stufe, a1 = Ereignis
    jal  ra, next_level      # a0 = Folgestufe
    mv   s0, a0              # Folgestufe speichern
    jal  ra, level_colour    # a0 = Farbwort
    sw   a0, 0(s3)           # LED-Pixel 0
    j    poll
```

Ein Team meldet: Nach dem Start mit Stufe 0 leuchtet die LED nach einem PLUS nur während einer einzigen Abfragegruppe grün, danach wieder schwarz. Im Simulator zeigt s0 dauerhaft 0.

1. Welche Folgestufe liefert `next_level` in der Gruppe mit PLUS? Welche liefert es in der folgenden Gruppe ohne Ereignis, wenn s0 noch 0 enthält? (1 P)
2. Welche Zeile des Blocks fehlt dem Team? Warum leuchtet die LED trotzdem eine Gruppe lang grün? (2 P)
3. Im fehlerhaften Aufbau steht nach einer Störung s0 = 17. Was zeigt die LED, und was bewirkt PLUS? Was stünde im korrekten Programm nach der nächsten Gruppe in s0? (1 P)

Musterlösung

1. In der PLUS-Gruppe liefert `next_level(0, 1)` die Stufe 1, in der folgenden Gruppe `next_level(0, 0)` wieder 0 (1 P).
2. Es fehlt `mv s0, a0` direkt nach `jal ra, next_level` (1 P). `level_colour` erhält die Folgestufe 1 unmittelbar in a0 und liefert Grün. Gespeichert wird sie nicht, deshalb rechnet die nächste Gruppe wieder mit 0 (1 P).
3. `next_level(17, ...)` liefert immer 0: Die LED bleibt schwarz, PLUS bleibt wirkungslos, s0 bleibt 17. Im korrekten Programm steht nach der nächsten Gruppe s0 = 0, auch ohne Ereignis (1 P).

Aufgabe 6: Ein Unterprogramm mit Stackframe (10 Punkte)

Ein zweites LED-Pixel soll als Vorschau zeigen, welche Farbe ein PLUS bewirken würde. Die Funktion `vorschau_farbe` erhält die Stufe in a0 und das Ereignis in a1 und liefert das Farbwort der Folgestufe in a0. Sie ruft dazu die Blattfunktionen `next_level` und `level_colour` auf; die Tabelle zeigt ihre sieben Befehle mit Adressen. Die Hauptschleife ruft sie mit `jal ra, vorschau_farbe` an Adresse `0x0040` auf. Beim Eintritt gelten s0 = 1, a0 = 1 (Stufe), a1 = 1 (PLUS), ra = `0x0044` und sp = `0xC000`.

1. Welches Farbwort liefert `vorschau_farbe` zurück? Ändert sich s0? (1 P)
2. Ergänzen Sie ra und sp nach jedem Befehl; die erste Zeile ist ein Beispiel. Befehle in `next_level` und `level_colour` fehlen, sie ändern ra und sp nicht. (3 P)

Adresse	Befehl	ra danach	sp danach
`0x0100`	`addi sp, sp, -16`	`0x0044`	`0xBFF0`
`0x0104`	`sw ra, 12(sp)`		
`0x0108`	`jal ra, next_level`		
`0x010C`	`jal ra, level_colour`		
`0x0110`	`lw ra, 12(sp)`		
`0x0114`	`addi sp, sp, 16`		
`0x0118`	`ret`		

3. Welchen Wert speichert `sw ra, 12(sp)` an welcher Adresse? Welche Byteadressen belegt das Wort? (1 P)
4. Warum reserviert die Funktion 16 Byte, obwohl sie nur vier braucht? (1 P)
5. Warum muss `vorschau_farbe` ra sichern, `next_level` aber nicht? (1 P)

Ein Team hat `sw ra, 12(sp)` und `lw ra, 12(sp)` weggelassen, um Speicherzugriffe zu sparen. Die übrigen fünf Befehle liegen nun ab `0x0100` lückenlos hintereinander. Symptom: Das Programm kehrt nie zur Hauptschleife zurück. Der PC wechselt endlos zwischen `0x010C` und `0x0110`, und sp wächst bei jedem Durchlauf um 16.

6. Notieren Sie die neuen Adressen der fünf Befehle. Welchen Wert hat ra beim ersten `ret`, und welcher Befehl hat ihn geschrieben? (1 P)
7. Erklären Sie die Endlosschleife und das Wachsen von sp. (1 P)
8. Ein Teammitglied schlägt vor, ra in t0 zu sichern: `mv t0, ra` am Anfang, `mv ra, t0` vor `ret`. Warum genügt das nicht? (1 P)

Musterlösung

1. `next_level(1, 1)` = 2, `level_colour(2)` = `0x00FF0000`, also Rot (0,5 P). s0 bleibt 1: Die Vorschau rechnet nur mit der Kopie in a0, s0 schreibt allein die Hauptschleife (0,5 P).

Adresse	Befehl	ra danach	sp danach
0x0104	`sw ra, 12(sp)`	0x0044	0xBFF0
0x0108	`jal ra, next_level`	0x010C	0xBFF0
0x010C	`jal ra, level_colour`	0x0110	0xBFF0
0x0110	`lw ra, 12(sp)`	0x0044	0xBFF0
0x0114	`addi sp, sp, 16`	0x0044	0xC000
0x0118	`ret`	0x0044	0xC000

2. Je Zeile 0,5 P. `ret` springt nach `0x0044` in die Hauptschleife.
3. Den Wert `0x0044` an `0xBFF0` + 12 = `0xBFFC`; das Wort belegt `0xBFFC` bis `0xBFFF` (1 P).
4. Die Aufrufkonvention verlangt einen auf 16 Byte ausgerichteten Stackzeiger; der Rest des Frames ist Füllung (1 P).
5. `jal ra, next_level` überschreibt ra mit `0x010C`; ohne Sicherung wäre der Weg zurück nach `0x0044` verloren. `next_level` ruft niemanden auf, sein ra bleibt bis zum `ret` erhalten (1 P).

Adresse	Befehl ohne Sicherung
0x0100	`addi sp, sp, -16`
0x0104	`jal ra, next_level`
0x0108	`jal ra, level_colour`
0x010C	`addi sp, sp, 16`
0x0110	`ret`

6. Adressen wie in der Tabelle. Beim ersten `ret` gilt ra = `0x010C`, geschrieben vom zweiten `jal` an `0x0108` (1 P).
7. `ret` springt nach `0x010C`. Dort erhöht `addi sp, sp, 16` den Stackzeiger erneut, danach folgt wieder `ret` mit unverändertem ra = `0x010C`. sp steigt so von `0xC000` auf `0xC010`, `0xC020` und weiter (1 P).
8. t0 ist ein temporäres Register; aufgerufene Funktionen dürfen es überschreiben. `level_colour` aus Aufgabe 8 verwendet selbst t0, die Rücksprungadresse wäre also verloren. Korrekt ist der Stackplatz wie im Original (1 P).

Aufgabe 7: Gerätezugriffe in Datenpfad und Pipeline (10 Punkte)

Die Hilfsfunktion `read_mask` liest die D-Pad-Register mit `lw`, die Hauptschleife schreibt das Farbwort mit `sw` und verzweigt mit `beq`.

1. Ergänzen Sie die Hauptsteuerung des Kursmodells für `lw t1, 4(a0)`, `sw a0, 0(s3)` und `beq a0, x0, release`. X steht für einen Wert ohne Wirkung. Die Spalte für `or t0, t0, t1` ist ein Beispiel. (3 P)

Signal	`or` (Beispiel)	`lw`	`sw`	`beq`
RegWrite	1			
ALUSrc	0			
MemRead	0			
MemWrite	0			
MemtoReg	0			
Branch	0			
ALUOp	10			

Zwei Fassungen des Anfangs von `read_mask` laufen im Fünf-Stufen-Kursmodell: IF, ID, EX, MEM, WB, Forwarding aus EX/MEM und MEM/WB, der Loadwert liegt erst nach MEM vor, die Pipeline ist anfangs leer. `andi` rechnet wie `addi` in EX.

Nr.	Fassung A	Fassung B
1	`lw t0, 0(a0)`	`lw t0, 0(a0)`
2	`andi t0, t0, 1`	`lw t1, 4(a0)`
3	`lw t1, 4(a0)`	`andi t0, t0, 1`
4	`andi t1, t1, 1`	`andi t1, t1, 1`

2. Zeichnen Sie das Stufenraster von Fassung A bis zum letzten WB; die Spalten sind die Takte. Markieren Sie gehaltene Stufen mit einem Stern, etwa ID*. Die erste Zeile ist vorgegeben. (3 P)

Nr.	1	2	3	4	5	6	7	8	9	10	11
1	IF	ID	EX	MEM	WB						
2											
3											
4											

3. Wie viele Stalls und wie viele Takte bis zum letzten WB braucht Fassung B? In welchem Takt und aus welchem Pipelineregister erhält Befehl 3 den Wert von t0? (2 P)
4. Beide Fassungen hinterlassen dieselben Werte in t0 und t1. Warum darf B umordnen? Betrifft der Unterschied die ISA oder die Mikroarchitektur? (1 P)
5. `read_mask` in Aufgabe 8 liest erst drei Register mit `lw` und maskiert danach mit drei `andi`. Wie viele Load-Use-Stalls entstehen dabei im Kursmodell? (1 P)

Musterlösung

Signal	`lw`	`sw`	`beq`
RegWrite	1	0	0
ALUSrc	1	1	0
MemRead	1	0	0
MemWrite	0	1	0
MemtoReg	1	X	X
Branch	0	0	1
ALUOp	00	00	01

1. Je Spalte 1 P. Dass die Adresse ein Gerät statt Speicher trifft, sieht die Hauptsteuerung nicht; sie entscheidet allein nach dem Befehl.

Nr.	1	2	3	4	5	6	7	8	9	10	11
2		IF	ID	ID*	EX	MEM	WB				
3			IF	IF*	ID	EX	MEM	WB			
4					IF	ID	ID*	EX	MEM	WB	

2. Je Zeile 1 P. Zwei Load-Use-Stalls, zehn Takte ($4 + 4 + 2$). Die Bubbles in EX4 und EX7 sind nicht verlangt. In EX5 und EX8 kommt der Loadwert jeweils aus MEM/WB.
3. Kein Stall, acht Takte (1 P). Befehl 3 erhält t0 in EX von Takt 5 aus MEM/WB, denn `lw t0` steht dann in WB (1 P). Ebenso erhält Befehl 4 in Takt 6 den Wert von t1 aus MEM/WB.
4. `lw t1` hängt nicht von `andi t0` ab und umgekehrt. Jeder Befehl hat dieselbe Wirkung wie vorher (ISA); nur der Zeitablauf in der Pipeline ändert sich (Mikroarchitektur) (1 P).
5. Keiner: Zwischen jedem `lw` und dem zugehörigen `andi` stehen zwei andere Befehle (1 P).

Aufgabe 8: Den Lüfterregler in Ripes testen

Das folgende Programm verbindet D-Pad, Ereigniserkennung und LED wie in Aufgabe 4. Farbwörter haben das Format `0x00RRGGBB`. Es fehlt nur der Rumpf von `next_level`.

```
.text
.globl main
main:
    li   s0, 0                  # Stufe
    li   s1, 0                  # Freigabe geschlossen
    li   s2, D_PAD_0_BASE
    li   s3, LED_MATRIX_0_BASE
poll:
    mv   a0, s2
    jal  ra, read_mask          # a0 = Rohmaske
    li   a1, 0                  # Ereignis: keines
    beq  a0, x0, release
    beq  s1, x0, update         # Freigabe geschlossen
    li   s1, 0                  # Freigabe verbraucht
    mv   a1, a0                 # 1: PLUS, 2: MINUS
    li   t0, 3
    bltu a0, t0, update
    li   a1, 3                  # STOP
    li   t0, 4
    beq  a0, t0, update
    li   a1, 0                  # Mehrfachdruck: keines
    j    update
release:
    li   s1, 1
update:
    mv   a0, s0
    jal  ra, next_level
    mv   s0, a0
    jal  ra, level_colour
    sw   a0, 0(s3)              # LED-Pixel 0
    j    poll

read_mask:                      # a0: Basis -> Rohmaske
    lw   t0, 0(a0)              # UP
    lw   t1, 4(a0)              # DOWN
    lw   t2, 8(a0)              # LEFT
    andi t0, t0, 1
    andi t1, t1, 1
    andi t2, t2, 1
    slli t1, t1, 1
    slli t2, t2, 2
    or   t0, t0, t1
    or   a0, t0, t2
    ret

level_colour:                   # a0: Stufe -> Farbwort
    li   t0, 1
    beq  a0, t0, colour_green
    li   t0, 2
    beq  a0, t0, colour_red
    li   a0, 0                  # schwarz
    ret
colour_green:
```

```
    li   a0, 0x0000FF00
    ret
colour_red:
    li   a0, 0x00FF0000
    ret

# Ihre Lösung aus Aufgabe 4: a0 = Stufe, a1 = Ereignis
next_level:
    ret                          # Platzhalter
```

1. Öffnen Sie Ripes, installiert oder im Browser unter ripes.dk, und wählen Sie über *Select Processor* den *RISC-V Single Cycle Processor*. Legen Sie im Tab *I/O* je ein D-Pad und eine LED-Matrix an; ein Doppelklick auf den Gerätenamen erzeugt das Gerät. Prüfen Sie in der Registerkarte des D-Pads die Offsets von UP, DOWN und LEFT (erwartet 0, 4 und 8) und bei den exportierten Symbolen `D_PAD_0_BASE` und `LED_MATRIX_0_BASE`. Protokollieren Sie Ripes-Version, Offsets und Basisadressen.
2. Geben Sie das Programm im Tab *Editor* ein und ersetzen Sie den Platzhalter durch Ihre Übersetzung der Tabelle aus Aufgabe 4.1, nur mit a- und t-Registern und ohne Stackframe. Tipp: Beginnen Sie wie `next_state` in der Vorlesung mit den Sonderfällen, etwa `li t0, 2` und `bgtu a0, t0, level_zero` für eine ungültige Stufe.
3. Setzen Sie einen Breakpoint auf den ersten Befehl bei `update`: ein Klick auf die blaue Leiste der rechten Programmansicht im Tab *Editor*. Starten Sie mit *Run* oder *Auto-clock*; beide halten am Breakpoint an. Bei jedem Halt stehen in a0 die Rohmaske, in a1 das Ereignis, in s1 die Freigabe und in s0 noch die alte Stufe. Betätigen Sie eine D-Pad-Taste, während das Programm hält, und setzen Sie den Lauf fort. Laut Vorlesung meldet das D-Pad einen Druck bis zum nächsten Lesen; prüfen Sie das an Ihrer Version.
4. Bedienen Sie nacheinander PLUS, PLUS, PLUS, MINUS und STOP mit mindestens einer Gruppe ohne Taste dazwischen. Protokollieren Sie je Gruppe a0, a1, s1, die neue Stufe und die LED-Farbe und vergleichen Sie mit Aufgabe 4.
5. Mehrere Tasten zugleich lassen sich mit der Maus schlecht betätigen. Stellen Sie deshalb mit PLUS die Stufe 1 ein, lassen Sie eine Gruppe ohne Taste folgen und setzen Sie einen zweiten Breakpoint auf `li a1, 0` nach `jal ra, read_mask`. Überschreiben Sie dort im Tab *Processor* den Wert von a0 durch einen Klick auf den Registerwert, in vier aufeinanderfolgenden Gruppen mit 6, 4, 0 und 4. Vergleichen Sie mit Aufgabe 4.2 und 4.3.
6. Ändern Sie die erste Zeile von `main` in `li s0, 17` und prüfen Sie s0 und LED nach der ersten Gruppe. Entfernen Sie danach zusätzlich `mv s0, a0` bei `update`, starten Sie mit `li s0, 0` neu und bedienen Sie PLUS. Vergleichen Sie mit Aufgabe 5.

Die LED zeigt nur die gewählte Stufe. Drei nacheinander gelesene Register sind keine gleichzeitige Momentaufnahme, und die Freigabe ersetzt keine Entprellung echter Taster.

Musterlösung

Eine mögliche Blattfunktion:

```
# a0 = Stufe, a1 = Ereignis -> a0 = Folgestufe
next_level:
    li   t0, 2
    bgtu a0, t0, level_zero     # ungültige Stufe
    li   t0, 3
    beq  a1, t0, level_zero     # STOP
    li   t0, 1
    beq  a1, t0, level_up       # PLUS
    li   t0, 2
    beq  a1, t0, level_down     # MINUS
    ret                         # kein Ereignis: bleibt
level_up:
    li   t0, 2
    beq  a0, t0, level_keep     # Obergrenze
    addi a0, a0, 1
    ret
level_down:
    beq  a0, x0, level_keep     # Untergrenze
    addi a0, a0, -1
level_keep:
    ret
level_zero:
    li   a0, 0
    ret
```

`bgtu` vergleicht vorzeichenlos; so landet auch ein negatives Muster wie `0xFFFFFFFF` bei `level_zero`.

Erwartung bei den Halten an `update`, Start mit Stufe 0 und geschlossener Freigabe; die Gruppen ohne Taste dazwischen liefern a0 = 0, a1 = 0 und s1 = 1:

Taste	a0	a1	s1	Stufe danach	LED
keine (erste Gruppe)	0	0	1	0	schwarz
PLUS	1	1	0	1	grün
PLUS	1	1	0	2	rot
PLUS	1	1	0	2	rot
MINUS	2	2	0	1	grün
STOP	4	3	0	0	schwarz

Mehrfachdruck bei Stufe 1 und offener Freigabe: a0 = 6 ergibt a1 = 0 und s1 = 0, die Stufe bleibt 1. Danach ergibt a0 = 4 kein Ereignis, a0 = 0 öffnet die Freigabe, und erst das nächste a0 = 4 erzeugt STOP (a1 = 3): Stufe 0, LED schwarz.

Mit `li s0, 17` steht nach der ersten Gruppe s0 = 0, die LED ist schwarz. Ohne `mv s0, a0` leuchtet die LED nach PLUS nur eine Gruppe lang grün, s0 bleibt 0. Das sind Modellvorhersagen; Version, Geräteadressen und tatsächliche Anzeige gehören ins Protokoll.